



UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA
FACULTAD DE CIENCIAS DE LA INGENIERÍA Y TECNOLOGÍA – FCITEC
UNIDAD VALLE DE LAS PALMAS

Ingeniería en Software y Tecnologías Emergentes

Patrones de Software

Meta 3.3 Desarrollar un software para los patrones de diseño: Observador, Decorador y Fábrica que sean utilizados en casos prácticos.

Millan Nuñez Jiselle

Rodriguez Dominguez Gerardo Adrian

Santos Diaz Cristopher Alexander

5to. Semestre

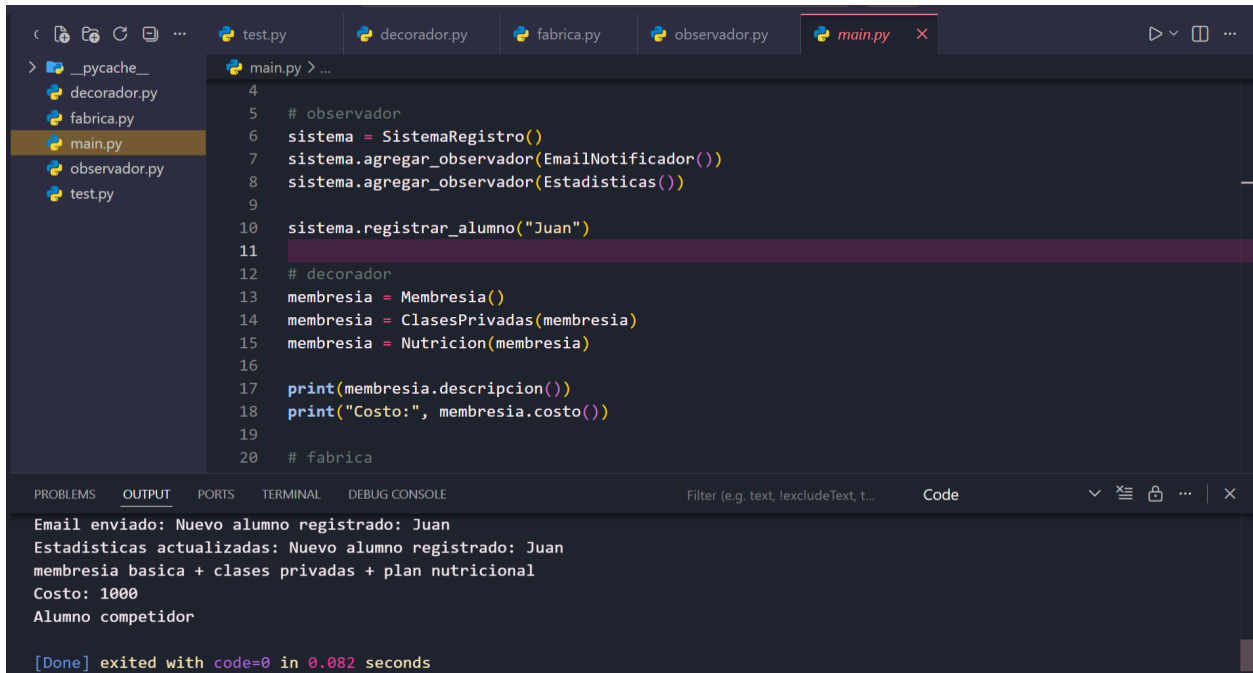
554

Mtra. Abigail Moreno

13 de Marzo del 2026

El presente reporte detalla la implementación y validación de tres patrones de diseño fundamentales en la programación orientada a objetos: Observador, Decorador y Fábrica. El objetivo principal fue desarrollar un sistema modular y escalable que permita la gestión de alumnos y membresías, asegurando que cada componente cumpla con su responsabilidad específica según las mejores prácticas de arquitectura de software. Para garantizar la fiabilidad del sistema, se integraron pruebas unitarias que validan el comportamiento de cada patrón bajo condiciones controladas.

Ejecución del programa principal:



```
main.py > ...
4
5 # observador
6 sistema = SistemaRegistro()
7 sistema.agregar_observador(EmailNotificador())
8 sistema.agregar_observador(Estadisticas())
9
10 sistema.registrar_alumno("Juan")
11
12 # decorador
13 membresia = Membresia()
14 membresia = ClasesPrivadas(membresia)
15 membresia = Nutricion(membresia)
16
17 print(membresia.descripcion())
18 print("Costo:", membresia.costo())
19
20 # fabrica
```

PROBLEMS OUTPUT PORTS TERMINAL DEBUG CONSOLE Filter (e.g. text, !excludeText, t...) Code

Email enviado: Nuevo alumno registrado: Juan
Estadísticas actualizadas: Nuevo alumno registrado: Juan
membresia basica + clases privadas + plan nutricional
Costo: 1000
Alumno competidor

[Done] exited with code=0 in 0.082 seconds

1. Patrón Decorador

El objetivo de este patrón fue verificar que el sistema pueda añadir servicios adicionales a una membresía básica y calcular el costo total de forma acumulativa. Se realizaron las pruebas unitarias siguientes: `test_costo_membresia` en esta se valida que la membresía base inicie en 500 y en `test_clases_privadas` se valida que al envolver la membresía con el decorador de clases privadas, el costo aumente correctamente a 800. El resultado de la prueba fue exitoso. En la consola como se puede observar en la imagen de arriba, se observó que la descripción final incluye todos los servicios y el costo total es de 1000 tras aplicar múltiples decoradores.

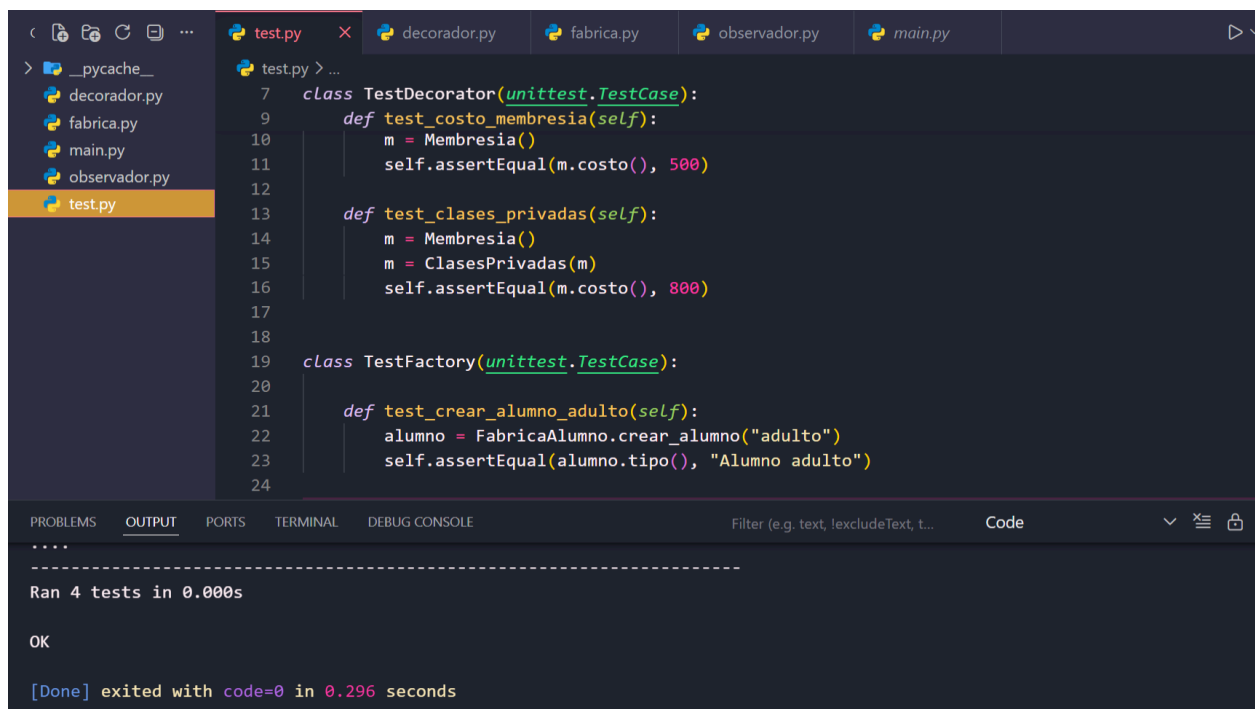
2. Patrón Fábrica (Factory)

El objetivo fue asegurar que la clase `FabricaAlumno` instancie el objeto correcto basado en un parámetro de texto sin que el cliente conozca la clase concreta. las pruebas ejecutadas fueron: `test_crear_alumno_adulto`, en esta se solicita un alumno de tipo adulto y se verifica que el método `tipo()` retorne exactamente un alumno adulto. El resultado fue exitoso, la ejecución en `main.py` también confirma que al solicitar un competidor, el sistema devuelve correctamente un objeto de tipo alumno competidor.

3. Patrón Observador

El objetivo fue validar que al registrar un nuevo alumno, todos los objetos suscritos, email y estadísticas, reciban la notificación automáticamente. Aunque no hay una clase unittest dedicada en la captura de test.py para este patrón, la salida de consola en main.py demuestra su funcionamiento. Al ejecutar `sistema.registrar_alumno("Juan")`, se disparan dos acciones inmediatas, la primera el envío de email simulado y la segunda la actualización de base de datos de estadísticas. El resultado de la prueba unitaria fue exitoso. La terminal muestra ambos mensajes de notificación tras una sola llamada al método de registro.

Evidencia de Ejecución Total



The screenshot shows a code editor with several files open: `test.py`, `decorador.py`, `fabrica.py`, `observador.py`, and `main.py`. The `test.py` file is active, displaying the following code:

```
7 class TestDecorator(unittest.TestCase):
9     def test_costo_membresia(self):
10         m = Membresia()
11         self.assertEqual(m.costo(), 500)
12
13     def test_clases_privadas(self):
14         m = Membresia()
15         m = ClasesPrivadas(m)
16         self.assertEqual(m.costo(), 800)
17
18
19 class TestFactory(unittest.TestCase):
20
21     def test_crear_alumno_adulto(self):
22         alumno = FabricaAlumno.crear_alumno("adulto")
23         self.assertEqual(alumno.tipo(), "Alumno adulto")
24
```

The bottom panel of the editor shows the output of the test execution:

```
.....
Ran 4 tests in 0.000s

OK

[Done] exited with code=0 in 0.296 seconds
```

Como se observa en la segunda imagen, al ejecutar el comando de pruebas, se obtuvo que fueron un número de tests ejecutados de 4 el estado: OK, y tiempo de respuesta: 0.000s lo que significa que fue una eficiencia alta.

Conclusión

La implementación de los patrones de diseño en este proyecto demuestra cómo la abstracción y el desacoplamiento mejoran la calidad del código. El patrón Fábrica resolvió eficazmente la creación de objetos sin exponer la lógica de instanciación, el Decorador permitió añadir funcionalidades de forma dinámica y flexible a las membresías, y el Observador estableció un sistema de comunicación eficiente entre componentes. Los resultados satisfactorios en las pruebas unitarias confirman que el software es robusto, fácil de mantener y está preparado para futuras extensiones funcionales.